

Report

Malaysia license plate holder

This is a report on the take-home assessment : “Malaysian License Plate Model”. The aim of this project is to train a model that first detects and second recognises Malaysian Car plates, and the given duration for this project is 3 days. This report outlines:

- I. How to run the code
- II. Results achieved
- III. Brief overview of model
- IV. Future works
- V. Deployment plans
- VI. Figures

How to Run the code

I recommend looking through the README in the Github repository as it gives a clearer idea of what to do. It is also a lot more detailed and easier to understand.

Installation (Repository)

Clone or extract the repository

- `git clone git@github.com:BeanBois/malaysia_carplate_take_home_assignment.git`
- `cd malaysian_lp`

Create virtual environment (recommended)

- `python -m venv venv`
- `source venv/bin/activate`

Install dependencies

- `pip install -r requirements.txt`

Installation (pre-Trained YOLOv8)

Install the weights [here](#) and name it 'yolov8n.pt' in current working directory

Training Datasets

Download dataset [here](#) and place under data/ (link should redirect you to 3 folders: "raw", "additional_data" and "negative_examples". Put theses folders under data/ . Dont change the folder structure!)

Testing Datasets

Download dataset [here](#) and place under testdata/ (link should redirect you to a folder called 'images'. put this 'images' folder under testdata/)

- To train model, run :

```
python detector.py --mode integrate --augment --epoch 100
```

Before testing, download the data at : [XYZ.com](#)

- To run test and inference, run :

```
python inference.py --directory testdata/images/ --model  
models/detection_training/plate_detector{n}/weights/best.pt  
--multi-engine
```

*** n depends on how many times you have trained the model. Basically locate the desired model under models/detection_training/***

*** I recommend copying the commands from README (and referring to it!) in GitHub since this is abit buggy due to fonts.**

Accuracy achieved

Dataset:

Total images: 28
Successfully processed: 28
Failed: 0

Detection Metrics:

Precision: 0.960
Recall: 0.800
F1 Score: 0.873
TP: 24, FP: 1, FN: 6

Recognition Metrics:

Character Accuracy: 0.262
Word Accuracy: 0.083
Avg Edit Distance: 3.75
Total Matched: 24

Firstly, Detection Module seems to be functional, seeing that it achieves a high Precision, Recall and F1 score. (Refer to Figure 1 & 2). It is even capable of detecting 2 carplates in one images! (Figure 3)

However, the Recognition Modules seems to be the one costing the model alot of mistakes, and it is evident that the reliance on a pre-trained and not fine-tuned OCR model has costed the model alot of mistakes. This is further supported by the visualisations (refer to Figure 4).

Brief overlook of Model

The model consist of a 2 stage pipeline consisting of:

- I. Detection Module (Locates license plates in images)
- II. Recognition Module (Reads text from detected plates)

Detection Module

The Detection Module is fine-tuned YOLOv8 model. YOLOv8 is utilised as it is considered the state of the art real-time object detection model and has a great balance of speed and accuracy. It is also easy to deploy and is capable of handling multiple scales and challenging conditions (weather, blur) well.

Within the detection module, a Multi-Stage Validation System(MSVS) is utilised for False Positive Filtering. This is so as real-time images usually contains text that resembles license plates (billboards, signs, truck markings). The MSVS checks for:

- Aspect Ratio: Plates usually have a fixed ratio (2.5:1 - 6:1)
- Size Validation: For detection to occur effectively, plates should occupy 0.5% - 15% of image area
- Texture Analysis: Plates have minimum contrast/variation

Recognition Module

The Recognition Module takes on a Multi-OCR (Optical Character Recognition) Engine Approach instead of a single OCR engine for robustness. It consists of:

- EasyOCR (Robust against varied lighting)
- PaddleOCR (Fast and lightweight)
- Tesseract (Good for high contrast)

and intelligently combines each engines results. We are not training the Recognition module as to save training time, and that pre-trained Recognition module should be sufficient for the task.

The Process flow of the Recognition Module is as follows. Firstly, plate image from the Detection Module is preprocessed. Then, multiple OCR engines are ran in parallel on the preprocessed image to extract text (for each engine). Extracted text (for each engine) are then cleaned and normalised (uppercase, remove special characters if present) and validated against Malaysian Plate formats (eg. ABC 1234) to induce priors into our models efficiently (speeds up training). The best result is then selected based on confidence and format match.

***Format validation has been removed as it somehow gave a worse performance due to inconsistency in Recognition module. However, ideally, this should be a functional sub-module.**

Preprocessing of images consists of several steps:

- Contrast Limited Adaptive Histogram Equalisation (CLAHE) is done for enhancement
- Image is demonised to remove artefacts
- Adaptive threshold in is done for better character separation
- Morphological operations are performed on images to connect broken characters

Dataset Sourcing and Preparation

Ideally, the dataset should consist of Malaysian carplates (True Positive) and common Malaysian Streetview (False Positive) that the model will commonly face as inputs during deployments. However there is limited labelled Malaysian license carplates data available publicly, and likewise for common Malaysian StreetView. As such, multiple approach are taken to address the issue of limited data.

Firstly, I removed the constraint of carplates being solely Malaysian, and mixed in carplates from multiple countries. Specifically, I mixed my dataset with Malaysian carplates (from Roboflow, 168 images) and Non-Malaysian carplates (from Kaggle, 400+ images). For false positives, I handpicked 60 images from Google Image, and they consists of Road signs, Billboards with texts, Store signs, Car bumpers without plates, Street views and empty parking lots. Data is then split into Training (70%), Validation (20%) and Test (10%) sets. Secondly, data is aggressively augmented not create variations, such that a single image can be used multiple times. This also adds diversity to our dataset, such that our model remains robust to different lighting, weather conditions and different camera angles.

To augment data, techniques listed below are applied:

- Horizontal flip
- Rotation (+/- 15 deg)
- Perspective transform
- Brightness/contrast Adjustment
- Motion blur, Gaussian blur

- Noise addition
- Weather effects

While more dataset can be included, due to limited time for training and limited computation available, I have to balance the trade-off between dataset size and training time.

Dataset for evaluation utilises the Malaysian carplates, and is hand-labelled by me.

Future Works (if more time is give)

Depending on the time given, certain improvements can be made to the model.

With an addition of 1-2 weeks, more data can be included given more training time, and better data augmentation () can be done. Preprocessing (Adaptive preprocessing) and Format validation (Adapt to non-Malaysian carplates during training, validation against 'released' car plates, state-based validation during deployment ect) can be further improved. Lastly, the entire code is very hard-coded (or tightly-coupled) and **only tested on MacOS**, and as such if given more time, I will want to remove some code debt from the file, decouple it and make it more robust to other systems.

With an addition of 1-2 months, a custom OCR model can be designed such that it specialises in license plate character only, as right now we are using pre-trained OCR model for recognition, without training (to save training time). The pre-trained OCR model is experimentally is the bottleneck for our performance, and as such, doing this can bring about big improvements to our model.

This can be done using CRNN architecture and fine-tuned specifically on Malaysian plates (or if there is limited data, carplates in general). An active learning pipeline can also be included such that difficult examples during deployment can be collected and used for training. This allows model to perform better continuously as it is being deployed and be more adept in Malaysia carplates detection. Object tracking can also be implemented (using ByteTrack) such that model is capable of aggregating predictions across multiple frames. This allows model to handle partial occlusions better and reduce false positives through temporal consistency.

With an addition of 3-6 months, an End-to-End model can be designed from scratch. This model can utilise more sophisticated architectures (Graphical-based architecture like GCNN, Transformer-based architecture like DETR), instead of being confined by less sophisticated pre-trained models. End-to-end learning with a more sophisticated model can allow for more 'direct' learning and eliminate pipeline overheads, resulting in better model performance. There will also be time to test and optimise the models on edge devices, such that they can run smoothly with little computation and storage. Deep research like Neuromorphical Learning (efficient and lightweight) and Few-shot learning (adaptive learning) can also be done for a more efficient and sophisticated model.

Deployment approach

I imagine this to be deployed in edge devices like cameras. To deploy the model, I will first outline the deployment architecture below:

1. Application Layer (Detection & Recognition Services, hosted locally)
2. API Gateway (Send results to central server)
3. Central server (Database & Monitoring)

For the Application Layer, I will utilise Docker for easy deployment and scaling. This is so as it ensures consistency across environments in different devices and isolates dependencies, allowing our model to be robust to different devices. Since these devices are deployed in edge devices, I will convert the trained YOLOv8 model to ONNX, which is more lightweight and faster, allowing them to be ran on limited computation and storage. Testing will be required and done on cameras to ensure accuracies is maintained after conversion. Each device will then run the model locally and store detection results if they are offline. Results from multiple devices will then be routinely sent to a central server using FastAPI (using REST API specifically), which stores them using PostgreSQL. Likewise, the central server will be a FastAPI server that is hosted using AWS. The database will store relevant data like timestamp, camera ID, detection logs and metadata (confidence score). A web dashboard can then be used to monitor detection logs and accuracies. It will also monitor the live-status of each running cameras, alerting client when a camera goes down.

Figures

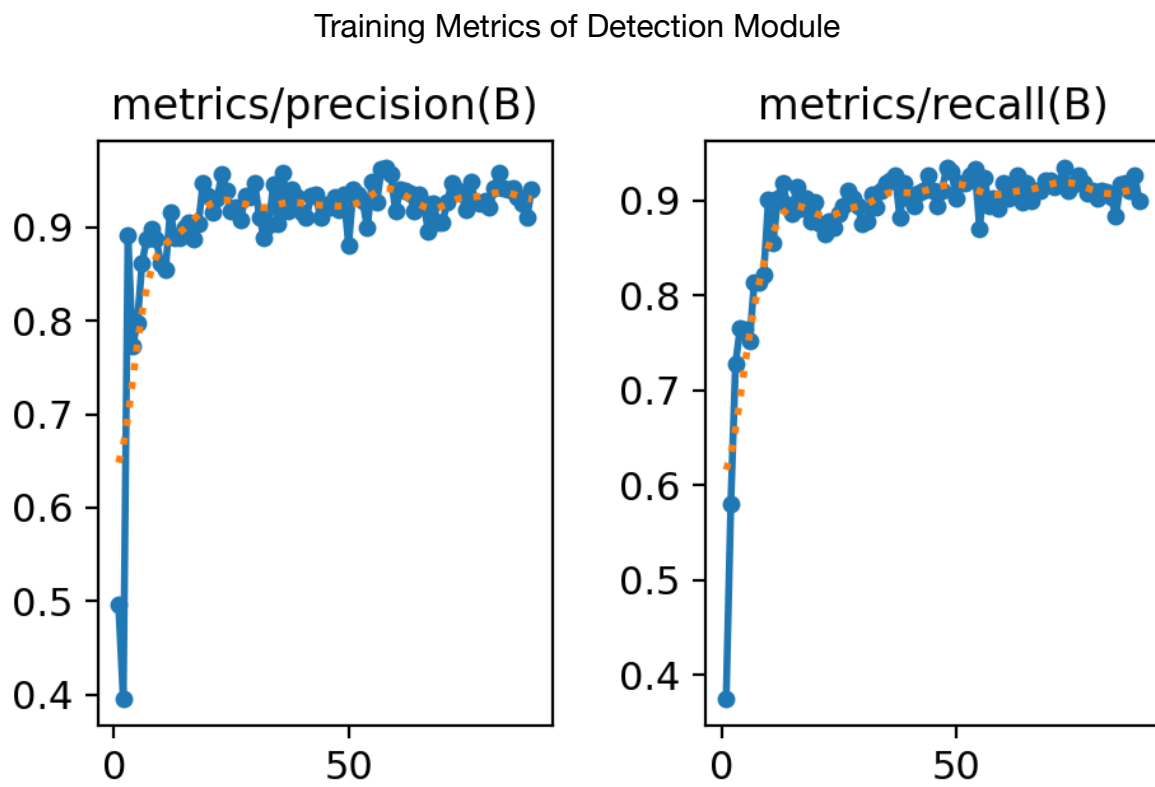


Figure (1)

Detected Carplates from Evaluation



Figure 2

Recognition Module detecting 2 carplates



Figure 3

Recognised carplates from Evaluations



Figure 4